ECSE 222, Digital Logic

VHDL 1

Fall 2023

University of McGill

Presented by:

Nara Yun (Student #: 261115268)

Karen Chen Lai (Student #: 261107674)

Instructors: Prof. Boris Vaisband

Date: September 19th, 2023

Executive Summary

We learned how to utilize the Intel Quartus II FPGA design software and to build the framework of the project through the step by step tutorial in this lab. We started by creating a new VHDL file by copying its entity codes for OR gate from an online source. Next, we created another new VHDL file for the testbench code. Finally, we ran and evaluated the EDA RTL Simulation and reviewed the schematic of the designed circuit under RTL representation, which is the gate-level representation of the design, after we have verified that the compile design has succeeded. On the other side, we created another new project for XNOR Gate by modifying a few parts of the code, we tested it by observing the RTL simulation and debugged the codes where it had problems with. Moreover, we learned how to read and analyze the RTL Simulations.

Screenshots of Schematics and Simulation Results

```
1   library IEEE;
2   use IEEE.std_logic_1164.all;
3
4   entity Nara_Yun_vhdl1 is
5   port(
6      a: in std_logic;
7      b: in std_logic;
8      q: out std_logic);
9   end Nara_Yun_vhdl1;
10
11  architecture rtl of Nara_Yun_vhdl1 is
12  begin
13     process(a, b) is
14     begin
15        q <= a or b;
16     end process;
17  end rtl;
18
```

Figure 1. OR Gate VHDL Code

```vhdl
library IEEE;
use IEEE.std_logic_1164.all;

entity testbench is
-- empty
end testbench;

architecture tb of testbench is

-- DUT component
component Nara_Yun_vhdl1 is
port(
  a: in std_logic;
  b: in std_logic;
  q: out std_logic);
end component;

signal a_in, b_in, q_out: std_logic;

begin

  -- Connect DUT
  DUT: Nara_Yun_vhdl1 port map(a_in, b_in, q_out);

  process
  begin
    a_in <= '0';
    b_in <= '0';
    wait for 1 ns;
    assert(q_out='0') report "Fail 0/0" severity error;

    a_in <= '0';
    b_in <= '1';
    wait for 1 ns;
    assert(q_out='1') report "Fail 0/1" severity error;

    a_in <= '1';
    b_in <= 'X';
    wait for 1 ns;
    assert(q_out='1') report "Fail 1/X" severity error;

    a_in <= '1';
    b_in <= '1';
    wait for 1 ns;
    assert(q_out='1') report "Fail 1/1" severity error;

    -- Clear inputs
    a_in <= '0';
    b_in <= '0';

    assert false report "Test done." severity note;
    wait;
  end process;
end tb;
```

Figure 2. OR Gate Testbench

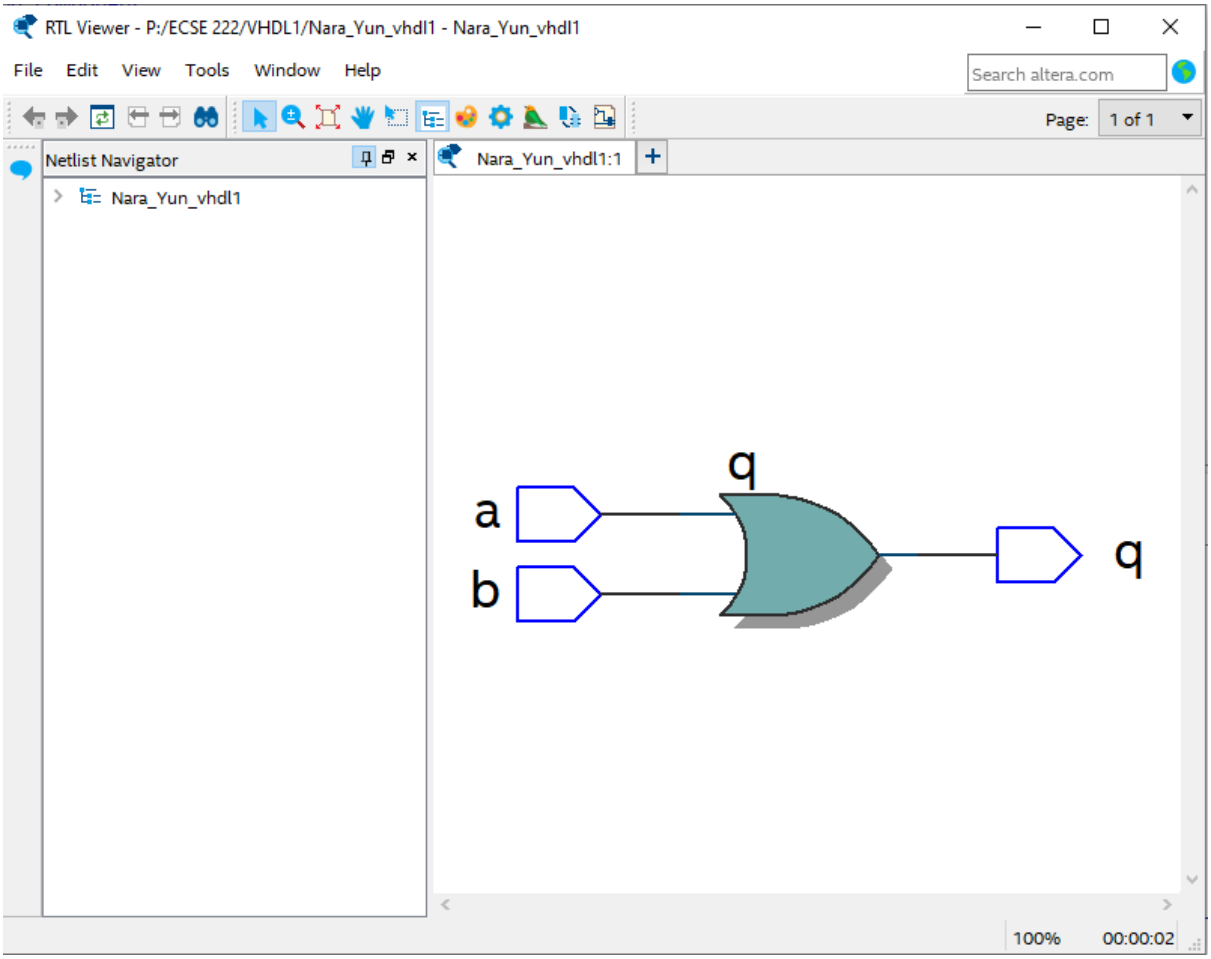

Figure 3. OR gate Schematic RTL View

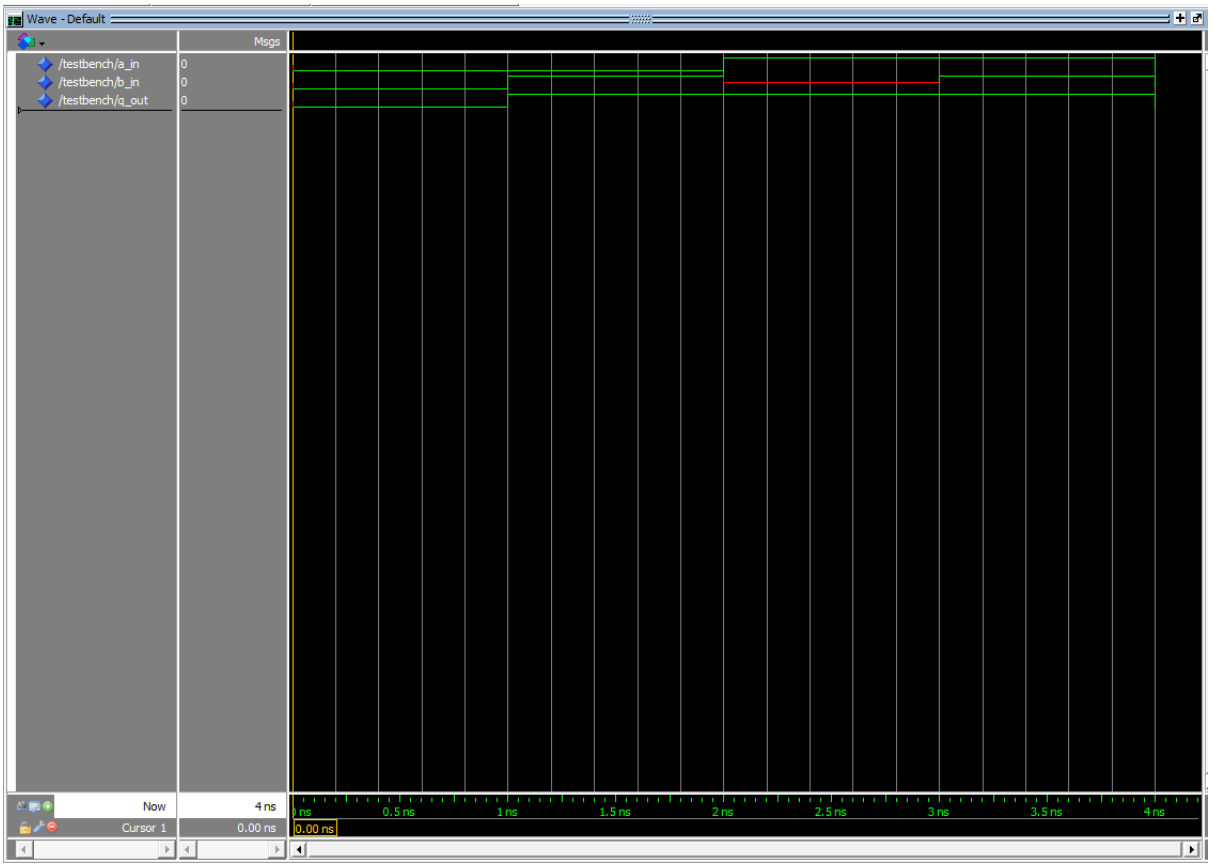

Figure 4. OR Gate RTL Simulation Result

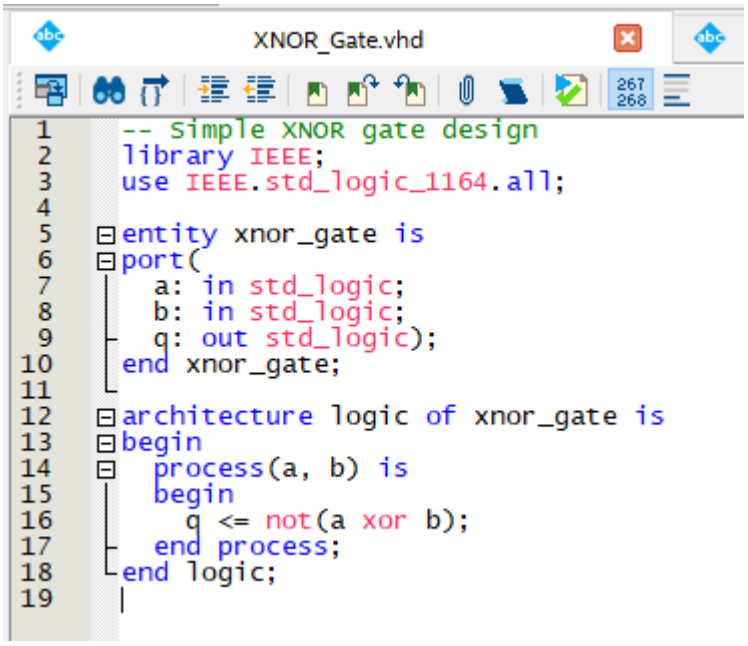

Figure 5. XNOR Gate VHDL Code

XNOR_Gate.vhd | XNOR_Gate_testbench.vhd

```vhdl
-- Testbench for XNOR gate
library IEEE;
use IEEE.std_logic_1164.all;

entity xnor_gate_testbench is
-- empty
end xnor_gate_testbench;

architecture tb of xnor_gate_testbench is

-- DUT component
component xnor_gate is
port(
  a: in std_logic;
  b: in std_logic;
  q: out std_logic);
end component;

signal a_in, b_in, q_out: std_logic;

begin

  -- Connect DUT
  DUT: xnor_gate port map(a_in, b_in, q_out);

  process
  begin
    a_in <= '0';
    b_in <= '0';
    wait for 1 ns;
    assert(q_out='1') report "Fail 0/0" severity error;

    a_in <= '0';
    b_in <= '1';
    wait for 1 ns;
    assert(q_out='0') report "Fail 0/1" severity error;

    a_in <= '1';
    b_in <= '0';
    wait for 1 ns;
    assert(q_out='0') report "Fail 1/0" severity error;

    a_in <= '1';
    b_in <= '1';
    wait for 1 ns;
    assert(q_out='1') report "Fail 1/1" severity error;

    -- Clear inputs
    a_in <= '0';
    b_in <= '0';

    assert false report "Test done." severity note;
    wait;
  end process;
end tb;
```

Figure 6. XNOR Gate Testbench

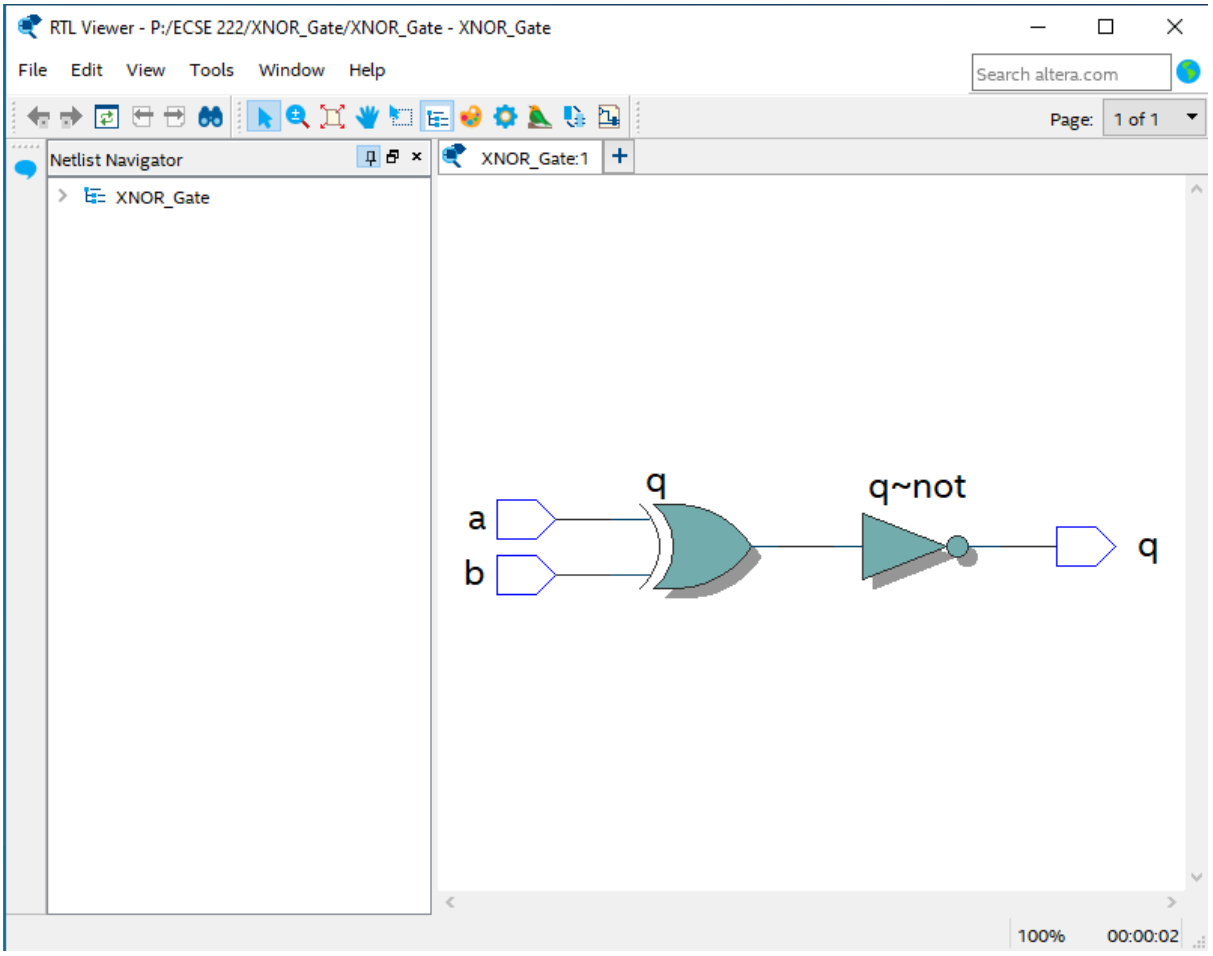

Figure 7. XNOR Gate Schematic RTL View

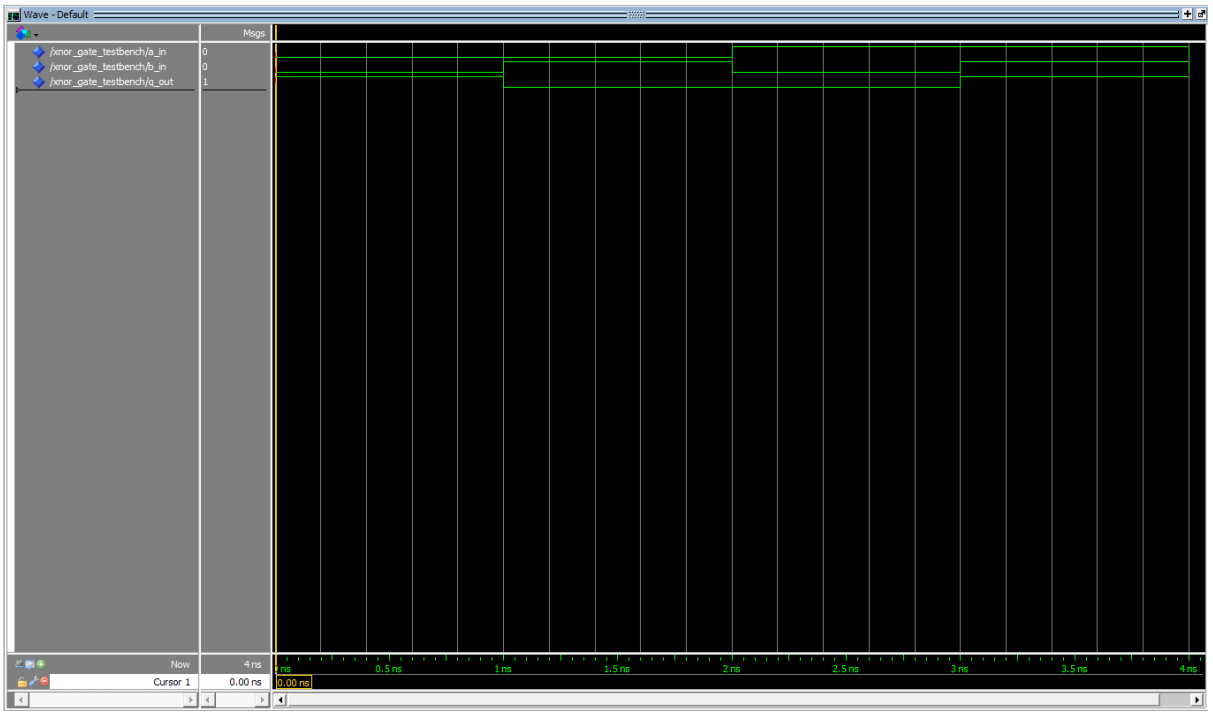

Figure 8. XNOR Gate RTL Simulation Result

Explanation of the Results

The OR gate implies logical disjunction, a + b, which returns the boolean constant 1, “true”, when at least one of its inputs is true; Otherwise, it returns 0, “false”.

For the OR gate, it demonstrates 4 different test cases that we have put in the testbench in the Figure 4. Firstly, the variables names are shown on the left hand side of the RTL simulation. It shows up the inputs “00”, “01”, “1x”, and “11” in the first two rows, and their respective outputs: 0, 1, 1 and 1 in the third row. It is also noticeable that different height levels of the green lines indicate different boolean constants, a green line in a higher level implies 1, whereas a green line in a lower level indicates 0. The red line in the third column indicates x, an unknown variable, which is not appeared in the XNOR gate RTL simulation in Figure 8 since we did not put any unknown variable in the testbench code as shown in Figure 6. Besides, Figure 3 displays the schematic OR gate with the inputs and output labeled.

The XNOR gate means exclusive nor, $\overline{a \oplus b} = a \odot b$, it returns “true” when all the input variables are the same and returns “false” when all the inputs are different. For the XNOR gate, Figure 8 illustrates the inputs and outputs variables that we have put in the testbench. Where the 2 inputs variables are “00”, “01”, “10”, and “11” and with their respective outputs: “1”, “0”, “0”, and “1”. The range of the green line is also observed at 1 ns which is the value that we have put in the testbench as shown in Figure 6. In addition, Figure 7 represents the schematic XNOR gate with two inputs, a and b, with one output q. In this figure, the variables pass through XOR gate first and then to a NOT gate, which is equivalent to a XNOR gate.

Conclusion

Through this laboratory experiment, we learned how to:

- Create a new VHDL file
- Set up the framework of the project
- Run the Intel Quartus software
- Modify testbench codes
- Analyze the logic functions, RTL simulation, and logic gates